

Auditoria Independente de Sistema — RADAR B3 PRO IA

Projeto: C:\Projetos Antigravity\TESTE_BRAPI **Versão declarada pelo sistema:** v3.1.0 · build 2026.08.31-1715 · environment: "Produção / B3 Realtime" **Base auditada:** HEAD ea7a2b7 (07/09/2026) · 18.254 linhas TS/TSX · 18 arquivos de teste · 123 testes **Data da auditoria:** 08/09/2026 **Método:** auditoria cega em duas fases. Fase 1 sem leitura de qualquer laudo anterior; Fase 2 de reconciliação. Todo achado numérico foi **executado**, não inferido por leitura — os motores foram extraídos e rodados com entradas escolhidas pelo auditor (`probe1.ts` , `probe2.ts` , `probe3.ts`).

1. Parecer executivo

Nota de integridade: 3,5 / 10.

O sistema tem um núcleo de engenharia genuinamente bom e um perímetro que o contradiz.

O núcleo bom é o motor de eleição de estruturas de opções (`cme-election.ts`). A matemática de travas verticais está **correta** — verifiquei crédito, perda máxima, break-even e retorno sobre risco contra cálculo independente e batem. O gate de IV foi de fato endurecido, os diagnósticos de bloqueio trazem números reais, e a busca de Iron Condor por produto cartesiano é bem construída. Não há um único `Math.random` no projeto. Isso é trabalho sério.

O problema é que esse núcleo é alimentado e cercado por **uma camada sistemática de números fabricados que a tela não distingue de números medidos**. Não são um ou dois descuidos: é um padrão recorrente de `?? valor` e `|| valor` que converte, silenciosamente, "a fonte falhou" em "eis o número". contei **mais de 14 constantes distintas** exercendo esse papel, incluindo três valores diferentes de ATR de fallback (0,50 / 0,14 / 2,5% do spot), três de HV (24,5 / 26,0 / 25,0) e dois de spot (R\$ 30,00 / R\$ 10,00), espalhados por arquivos que deveriam compartilhar a mesma regra.

Três achados isolados bastariam, sozinhos, para reprovar o sistema para uso em alocação de capital real:

1. **Uma empresa com prejuízo de R\$ 5 bilhões e margem líquida de -12% é classificada APROVADO com score 80/100** — executado e reproduzido. Ambas as flags eliminatórias são silenciadas por um único campo (FCO) e o ROE exibido ao usuário é uma constante do código (18%), não uma medição.
2. **Quando o feed de cotações globais falha, o sistema exibe preços fixos gravados no código** (SPY 761,78 · Brent 94,37 · Ouro 4.321,59 · PETR4 46,87 +4,11%) sem qualquer marca de que são fallback, e calcula o Termômetro de Sentimento em cima deles.
3. **O break-even do Iron Condor exibido em tela é o ponto de LUCRO MÁXIMO**, não um break-even. O usuário lê "Break Even: R\$ 30,00" exatamente onde o payoff é +R\$ 0,80.

Some-se a isso a ausência total de barreiras automáticas — **sem ESLint, sem CI, sem etapa de lint, sem tipo de proveniência de dado** — e o quadro é de um sistema cuja qualidade depende inteiramente de revisão humana caso a caso, num domínio onde o erro custa dinheiro do usuário.

Recomendação: não usar em decisão de capital real no estado atual. Remediar, não reescrever — o núcleo justifica a remediação. Detalhe na Seção 8.

2. Matriz de achados

#	Severidade	Achado	Arquivo:linha
C1	● CRÍTICO	Flags eliminatórias LUCRO_NEGATIVO e MARGEM_NEGATIVA neutralizadas por FCO positivo	fundamentals.ts:114-119, 164-178, 309-321
C2	● CRÍTICO	ROE "normalizado" é uma constante fabricada (teto 18,0) que vira +15 pts	fundamentals.ts:209-211
C3	● CRÍTICO	Preços de mercado global gravados no código como fallback silencioso	market-quotes.ts:63-76
C4	● CRÍTICO	Break-even do Iron Condor exibido é o ponto de lucro máximo	cme-election.ts:1476
C5	● CRÍTICO	DL/EBITDA da VALE3 fixada em 0,8x por regra hardcoded de ticker	brapi.ts:326-335
C6	● CRÍTICO	Spot fabricado (R\$ 30,00 e R\$ 10,00) quando a cotação falha	options-barriers.ts:140 · cme-election.ts:346
C7	● CRÍTICO	Call/Put Wall inventadas em $\pm 5\%$ do spot quando não há posições	options-barriers.ts:412-423
C8	● CRÍTICO	HV21/HV63 fabricadas (24,5 / 26,0 / 25,0) e exibidas como medidas	options-barriers.ts:142-143 · quote/route.ts:160 · options/route.ts:102 · trends/route.ts:79
C9	● CRÍTICO	Falha na API de fundamentos vira veredito REPROVADO → recomendação de VENDA	quote/route.ts:111-113 · trends/route.ts:62-64
A1	● ALTO	Rastreador usa RSI fixo = 50 e ignora o gate de volatilidade	trends/route.ts:79, 105-117
A2	● ALTO	Painel setorial inteiro é hardcoded com percentuais fictícios	TradingViewOverview.tsx:160-200
A3	● ALTO	Pilares 4 e 5 do Termômetro afirmam usar DI e fluxo estrangeiro; não usam	sentiment.ts:270-283
A4	● ALTO	"Consultor IA" é um if/else de palavras-chave com respostas fixas	api/ai-consultant/route.ts:42-413
A5	● ALTO	UI declara Black-Scholes-Merton; gregas exibidas vêm de Cox-Ross-Rubinstein	OptionsBarriersView.tsx:299 vs brapi.ts:453
A6	● ALTO	percentile: 75 da IV é constante	options-barriers.ts:376
A7	● ALTO	Contador de visitantes chaveado em header controlado pelo cliente	analytics/track/route.ts:63-65
A8	● ALTO	E-mails de assinantes versionados em repositório com remote público	data/subscribers.json · newsletter/route.ts:40
A9	● ALTO	Sem ESLint, sem CI, sem etapa de lint; npm run lint não roda	raiz do projeto
A10	● ALTO	Zero autenticação e zero rate limit em todas as 8 rotas de API	src/app/api/**
A11	● ALTO	R:R de calculateRiskReward é tautológico: sempre 1,5	indicators.ts:279-283
A12	● ALTO	Item de checklist "Assimetria R:R ≥ 1.4 " não checa R:R algum	indicators.ts:219, 246-251
A13	● ALTO	ATR de fallback R\$ 0,50 aplicado a qualquer preço de ação	indicators.ts:116 · trade-plan.ts:86
A14	● ALTO	volumeRatio de fallback = 100% faz o item de volume PASSAR	indicators.ts:139 · study-auditor.ts:111,116
A15	● ALTO	black-scholes.ts sem nenhum teste; theta de PUT errado em 65%	black-scholes.ts:85-88
M1	● MÉDIO	Selic hardcoded 10,75% (vigente: 14,50%) e dois valores no código	black-scholes.ts:47 vs brapi.ts:465

#	Severidade	Achado	Arquivo:linha
M2	MÉDIO	Sessão de mercado por hora local do servidor, sem calendário B3	sentiment.ts:74-115
M3	MÉDIO	Thresholds duplicados fora de <code>rules.ts</code> (3,0% · 1,8 · RSI 40-65)	options-barriers.ts:425 · fundamentals.ts:116 · indicators.ts:216
M4	MÉDIO	Testes tautológicos: asserção sobre fixture e sobre texto de arquivo	v3-specification.test.ts:47-118 · api-contract.test.ts
M5	MÉDIO	Teste F8 consagra os números fabricados (0,8x e 18%) como esperados	fundamentals.test.ts:146-169
M6	MÉDIO	Regras de NORMALIZATION declaradas em <code>rules.ts</code> e nunca aplicadas	rules.ts:45-52
M7	MÉDIO	<code>environment: "Produção / B3 Realtime"</code> contradiz o próprio manual (D-1)	version.ts:8
M8	MÉDIO	Datas de vencimento e build hardcoded (2026-09-18, 31/08/2026)	options-barriers.ts:89,123 · version.ts
B1	BAIXO	<code>uniqueToday: Math.max(len, 1)</code> — piso cosmético no contador	analytics/track/route.ts:83
B2	BAIXO	<code>calculateHistoricalRSI</code> é $O(n^2)$ — recalcula do zero a cada ponto	indicators.ts:44-55
B3	BAIXO	ATR chamado de "ATR(14) padrão Wilder" mas é média simples	indicators.ts:113-133

Não são achados (verificado e aprovado): matemática de travas verticais (crédito, perda máxima, break-even, retorno); `assertDirection` como validação de coerência; solver de IV por bisseção (converge com erro < R\$ 0,005 e sensibilidade de 0,4 pp de vol por centavo — a exibição com 1 casa é honesta); `calculateMaxPain`; `normalCDF` (erro < $1,5 \cdot 10^{-7}$); ausência total de `Math.random`; ausência de segredo versionado (`.env.local` nunca entrou no git).

3. Críticos — com as provas executadas

C1 · Prejuízo de R\$ 5 bi e margem –12% resultam em APROVADO 80/100

`fundamentals.ts:116` testa `fcoVal >= 1.8 * netIncomeVal`. Quando `netIncomeVal` é **negativo**, `1.8 × negativo` é negativo, e qualquer FCO positivo satisfaz a condição — **sempre**. O sistema então classifica o prejuízo como "distorção por baixa não-caixa" e, nas linhas 164-178 e 309-321, **deixa de empilhar** `LUCRO_NEGATIVO` e `MARGEM_NEGATIVA`.

Prova executada (`probe1.ts`, P4), empresa fictícia com prejuízo de R\$ 5 bi, margem –12%, ROE 3%, FCO +R\$ 6 bi:

```
score = 80 | status = APROVADO | flags eliminadoras = []
ROE exibido = 18% (Norm.) | ROE contabil informado = 3%
resumo: "Ativo APROVADO no crivo fundamentalista CNPI-P com Score de 80/100 ..."
```

Mesma empresa, **um único campo alterado** (`operatingCashFlow: null`):

```
score = 65 | status = REPROVADO | flags = ["LUCRO_NEGATIVO", "MARGEM_NEGATIVA"]
```

Como se manifesta: o ativo entra na lista ALTA · COMPRA do rastreador e recebe trava de alta com strikes. O manual embutido no próprio sistema (`ai-consultant/route.ts:165`) promete ao usuário que `LUCRO_NEGATIVO` reprova "empresa que destrói capital". Na prática, basta ter caixa operacional positivo.

O que fazer: trocar por `netIncomeVal > 0 && fcoVal >= RULES.FCO_TO_NET_INCOME_DISTORTION_RATIO * netIncomeVal`. Normalização por não-recorrente exige o valor do item não-recorrente (`nonRecurringItems`), que a BRAPI não fornece — sem esse dado, a flag eliminatória deve prevalecer.

C2 · O ROE normalizado é uma constante, não uma medição

`fundamentals.ts:209-211`:

```
: (normalizedRoeVal ?? Math.min(18.0, Number((rawRoeVal * (fcoVal / Math.max(1, netIncomeVal || 1))))).to
```

Com lucro negativo, `Math.max(1, netIncomeVal) = 1`, o multiplicador vira o FCO inteiro em reais, e `Math.min` **grampeia o resultado exatamente em 18,00**. Na prova P4c o ROE exibido é 18% (Norm.) — o valor do teto, não uma propriedade da empresa. $18\% \geq 10\% \Rightarrow$ **+15 pontos** e descrição "ROE Normalizado ex-impairment reflete a capacidade de retorno do capital".

Não existe metodologia CNPI-P, ANBIMA ou CFA que normalize ROE escalando-o pela razão FCO/Lucro. **O que fazer:** remover o ramo. Se `normalizedRoe` não vier da fonte, exibir o ROE contábil ou N/D.

C3 · Preços de mercado gravados no código, exibidos como cotação

`market-quotes.ts:63-76` mantém uma tabela de fallback. O `try/catch` que a antecede engole **qualquer** falha do Yahoo Finance (403 a IP de datacenter, timeout de 4 s, rate limit) e a função **nunca lança**:

```
fetchLiveMarketQuote("SPY") = {"price":761.78,"change":-5.27,"changePct":-0.69, ...}
fetchLiveMarketQuote("ZZZZ") = {"price":100,"change":0,"changePct":0, ...} ← ticker inexistente
Campos do objeto: symbol, name, price, change, changePct, timestamp
getLiveMarketOverview() lançou erro? NÃO — retornou objeto "ao vivo"
```

Não existe campo de proveniência. A rota `/api/sentiment` recebe esse objeto como `liveOverview`, e o Termômetro é calculado normalmente:

```
score = 44 | temperatura = 23.2C | zona = CAUTELA & AVERSÃO A RISCO (FRI0)
cotações exibidas: DXY 99.79 (+0.14%) · EWZ 36.57 (+1.50%) · FEF1! 97.90 (-1.61%) · UKOIL 94.37 (-0.87%)
```

Todos fabricados. O cabeçalho do arquivo diz "Cotações Globais [...] em Tempo Real". A tabela ainda inclui `VALE3.SA: 78,30` e `PETR4.SA: 46,87 (+4,11%)`.

O que fazer: `fetchLiveMarketQuote` deve retornar `null` na falha; `CachedQuote` deve ganhar `source: 'LIVE' | 'STALE'` com data do último dado real; o Termômetro deve se recusar a pontuar pilar sem dado e a UI exibir a lacuna.

C4 · O break-even do Iron Condor é o ponto de lucro máximo

`cme-election.ts:1476`: `breakEven: (shortPut.strike + shortCall.strike) / 2`. Isso é o centro da faixa de lucro. Um Iron Condor tem **dois** break-evens: `K_put_vendida - crédito` e `K_call_vendida + crédito`.

Prova P8, condor 26/28/32/34 com crédito de R\$ 0,80:

```
breakEven EXIBIDO pelo sistema = 30
payoff nesse "break even"      = 0.8 ← é o LUCRO MÁXIMO, não zero
break-evens REAIS              = 27.20 e 32.80 -> payoff neles: -0 / 0
```

Como se manifesta: o operador lê "Break Even: R\$ 30,00" e conclui que abaixo de R\$ 30 a operação perde — quando é exatamente onde ela mais ganha. Dimensionamento e regra de saída

construídos sobre esse número saem errados nos dois sentidos. O campo é `breakEven: number` (singular) no tipo, então a correção exige mudança de contrato: `breakEvenLower` / `breakEvenUpper`.

C5 · Solvência da VALE3 escrita à mão

`brapi.ts:326-335`:

```
if (cleanSymbol === 'VALE3') {
  if (!financialDebtToEbitda || financialDebtToEbitda > 2.5) {
    financialDebtToEbitda = financialDebt && totalCash && ebitda && ebitda > 0
      ? Number(((financialDebt - totalCash) / ebitda).toFixed(2))
      : 0.8;
  }
}
```

Um ticker nominal com tratamento próprio. Quando o balanço não vem (o caso mais comum — `loansAndFinancing` frequentemente ausente), `financialDebtToEbitda` vira **0,8**. Isso é $\leq 2,5x \Rightarrow$ **+20 pontos**, status BOM, e a tela exibe `0.8x (Financ.)` com a descrição "*Alavancagem financeira líquida de 0,8x EBITDA (excluídas provisões de balanço)*" e o selo `RECONCILIADO_FINANCEIRO`. Um número inventado apresentado como resultado de reconciliação metodológica.

Agravante: o ramo `?` : recomputa **a mesma expressão** que acabou de ser rejeitada por ser $> 2,5$ — se havia dado, o valor não muda; o único efeito real do bloco é injetar o 0,8.

O que fazer: apagar o bloco inteiro. Reconciliação de IFRS-16 e provisões de descomissionamento exige a nota explicativa do ITR — não cabe em constante.

C6 e C7 · Preço e muralhas inventados quando a fonte falha

`options-barriers.ts:140`: `const spot = underlyingPrice > 0 ? underlyingPrice : 30.0;` `cme-election.ts:346`: `const spot = spotPrice > 0 ? spotPrice : optionAnalysis?.underlyingPrice || 10.0;`

Dois spots fictícios diferentes no mesmo caminho de execução. A rota `/api/options:44` é o gatilho: no `catch` da cotação ela monta `regularMarketPrice: 0`, e `0` cai direto no fallback. Diferente de `/api/quote` e `/api/trends`, essa rota ainda chama `analyzeOptionPositions` **mesmo com** `positions: []` (linha 89), o que aciona `buildOptionBarrierAlert` sobre nada.

Prova P5, com preço 0 e sem posições nem analytics:

```
underlyingPrice (spot) = 30      <- preco real era 0
hv21 = 24.5 | hv63 = 26          <- sem nenhum historico
topCallWall = {"strike":31.5,"symbol":"CALL","contracts":0,"distSpot":5}
topPutWall  = {"strike":28.5,"symbol":"PUT","contracts":0,"distSpot":-5}
```

Muralhas institucionais em R\$ 31,50 e R\$ 28,50 com **zero contratos**, para um ativo cujo preço é desconhecido. O usuário lê "Call Wall (Teto Institucional)". O `symbol` literal `'CALL'` é o único indício, e não aparece rotulado como erro.

C8 · Volatilidade histórica fabricada em quatro pontos do código

`options-barriers.ts:142-143` (`?? 24.5` e `?? 26.0`) e três rotas com `?? 25.0`. `calculateHistoricalVolatility` faz a coisa certa — retorna `null` sem histórico suficiente (P3 confirma) — e o chamador desfaz isso.

Pior: `volatility.ts:51-52` repete o padrão dentro do classificador:

```
classifyVolatilityRegime(0, 0) = {"regime":"MODERADA","ivRatio":1,
  "label":"VOLATILIDADE MODERADA / EQUILIBRADA","isCreditFavorable":true, ...}
```

Sem um único dado, o sistema afirma regime MODERADO, razão IV/HV de **1,00** e `isCreditFavorable: true` — que é o critério que autoriza venda de prêmio. Nenhum teste cobre esse caminho.

A UI expõe o resultado sem ressalva: `OptionsBarriersView.tsx:262` renderiza `{data.hv21}% / {data.hv63}%` cru, e a linha 299 grava **de novo** o 24,5: `HV21 ({data?.hv21 || 24.5}%)`.

C9 · Indisponibilidade da API vira recomendação de VENDA

`quote/route.ts:111-113` e `trends/route.ts:62-64`:

```

} catch {
  fundamentals = analyzeFundamentals(cleanSymbol, {});
}

```

Além disso `brapi.ts:356-357` já engole a exceção e devolve `{}`, de modo que o `catch` sequer é o caminho principal. Com objeto vazio: `score 0`, `status: 'REPROVADO'`.

Encadeando com `operation-matrix.ts:80` — **BAIXA + REPROVADO ⇒ VENDA**, `listedInTracker: true` — uma indisponibilidade da BRAPI em um ativo em tendência de baixa produz um ativo **listado na lista VENDA do rastreador**, com trava de baixa sugerida e alerta de aluguel BTC. O `summary` menciona "ausência de dados mínimos", mas o `status` que governa toda a máquina de decisão é REPROVADO, indistinguível de reprovação real.

O que fazer: introduzir um terceiro estado `INDISPONIVEL` e fazê-lo bloquear qualquer listagem, em vez de reaproveitar REPROVADO.

4. Segurança e proveniência

Rotas. As 8 rotas (`/quote`, `/trends`, `/options`, `/opportunities`, `/sentiment`, `/ai-consultant`, `/newsletter`, `/analytics/track`) não têm autenticação, autorização nem rate limit. `/trends?limit=all` dispara **200 análises em paralelo**, cada uma com 3 chamadas à BRAPI — 600 requisições autenticadas com o token do dono por requisição HTTP anônima. É um amplificador de custo e um caminho direto para esgotar a cota da API a partir de fora.

Rate limit chaveado em dado do cliente. `analytics/track/route.ts:63-65` deriva a identidade do visitante de `x-forwarded-for/x-real-ip`. Sem proxy confiável à frente que sobrescreva o header, qualquer cliente escolhe o próprio hash rotacionando o valor e infla o contador de "visitantes únicos" arbitrariamente. Agravantes: é um **GET que muda estado** (portanto pré-buscável e cacheável), o array `uniqueHashes` cresce sem limite, e `uniqueToday: Math.max(len, 1)` estabelece um piso cosmético de 1.

Segredos. Aprovado. `BRAPI_API_KEY` vive só em `.env.local`, coberto por `.gitignore`, **nunca commitado** (`git log --all -- .env.local` vazio). Não há prefixo `NEXT_PUBLIC_`, e a chave só é usada em `route handlers` do servidor — não vaza para o bundle. `.env.local.example` contém apenas o placeholder.

Dado pessoal versionado. `data/subscribers.json` e `data/analytics.json` **estão rastreados no git**, e o remote é `https://github.com/jrveloso8-ai/RADARB3.git`. Hoje o arquivo tem 33 endereços de teste, mas `newsletter/route.ts:40` grava o e-mail real de qualquer inscrito nesse mesmo arquivo e `publicar_github.bat` executa `add/commit/push` do diretório. É um pipeline que empurra e-mail de terceiro para repositório remoto — LGPD art. 46. Adicione `/data` ao `.gitignore`, remova do índice (`git rm --cached`) e mova a persistência para fora do repositório.

Barreiras automáticas — todas ausentes.

Barreira	Estado
ESLint configurado	✗ nenhum <code>.eslintrc*</code> / <code>eslint.config.*</code> ; <code>eslint</code> nem está em <code>devDependencies</code>
Etapa de lint no CI	✗ não existe <code>.github/</code> — não há CI
Tipos de proveniência	✗ nenhum campo <code>source</code> / <code>isFallback</code> em <code>HistoricalPrice</code> , <code>CachedQuote</code> ou <code>OptionAnalysisResult</code>
Pasta de fontes com payload real	⚠ parcial — existem 4 fixtures em <code>src/lib/services/__fixtures__/</code> , mas não há <code>docs/fontes/</code> com payload bruto datado

`npm run lint` está declarado no `package.json` apontando para `next lint`, que sem configuração não roda. Os imports mortos que encontrei (`calculateBlackScholes` e `calculateImpliedVolatility` em `options-barriers.ts:20`, `classifyVolatilityRegime` em `trends/route.ts:11`, `isActionableStrategy` em `OptionsBarriersView.tsx:24`) são justamente o que um `no-unused-vars` pegaria — e o terceiro deles é a assinatura de um defeito real (Seção 7).

5. Rótulo versus implementação

O sistema afirma capacidades que o código não entrega. Listados por gravidade da afirmação:

"Consultor IA" / "Chat IA especializado" (`api/ai-consultant/route.ts`). Não há IA. São 12 blocos `if (q.includes('...'))` devolvendo markdown fixo; o `GET` da rota se anuncia como `service: "Agente de IA do Manual Radar B3 v2"`. Qualquer pergunta fora das palavras-chave cai numa resposta genérica que ecoa a própria pergunta. Como base de conhecimento navegável é útil e honesta; como "IA" é uma afirmação falsa.

"Modelo Matemático: Black-Scholes (Merton 1973 c/ q=DivYield)" (`OptionsBarriersView.tsx:299`). Nenhuma grega ou IV exibida passa por esse modelo. Verifiquei todas as chamadas: de `black-scholes.ts` só `calculateMaxPain` é usado em produção; `calculateBlackScholes` e `calculateImpliedVolatility` são código morto. As gregas e a IV vêm da BRAPI, e `brapi.ts:453` registra o modelo real da fonte: `'cox-ross-rubinstein'` — binomial, não Black-Scholes.

"Curva de Juros & Risco Fiscal Brasil (DI Futuro / EWZ)" e "Fluxo Institucional & Estrangeiro na B3" (`sentiment.ts`, pilares 4 e 5, somando **45% do peso** do Termômetro). Não há um único dado de DI nem de fluxo estrangeiro no projeto. Ambos os scores derivam apenas do sinal de variação do EWZ e do SPY. O resumo do pilar 5 é uma **string constante**: *"Saldo de capital externo na B3 e fluxo em ADRs sustentam suporte no mercado à vista"* — afirmada sempre, com qualquer dado.

Painel setorial (`TradingViewOverview.tsx:160-200`). Nomes de setor, variação (`+0.25%`), peso no índice (`14.2%`), líderes com percentuais (`VALE3 (+0.3%)`) e comentário editorial, todos literais no código. Um deles diz *"barril Brent acima de US\$ 78"* enquanto o módulo de sentimento do mesmo sistema usa Brent a 94,37.

environment: "Produção / B3 Realtime" (`version.ts:8`). O próprio manual do sistema, tópico 11, diz o contrário: *"Opera com dados de fechamento em opções: posições em aberto e IV são do pregão anterior"*. `build` e `releaseDate` congelados em 31/08/2026, com commits até 07/09.

"Assimetria Suporte vs Resistência (R:R >= 1.4)" (`indicators.ts:246-251`). O item passa quando `isMaAligned && isMacdPositive` (linha 219) — médias e MACD. Nada de suporte, resistência ou R:R entra na avaliação.

`calculateSupportResistance` promete "Suportes e Resistências Dinâmicos e Estáticos", mas o primeiro suporte e a primeira resistência são `spot × 0,98` e `spot × 1,02` (linhas 167 e 171) — percentuais fixos, não estrutura de preço. E é o `supports[0]` (`±2%`) que `calculateRiskReward` usa como stop.

6. Testes: 123 verdes que não pegam nada disto

A suíte roda em 4,6 s, 18 arquivos, **123 testes, 123 verdes** — e não detecta um único dos 9 críticos. Os motivos são estruturais:

Tautologias por asserção sobre a fixture. `v3-specification.test.ts:94-118` — CT-952 afirma que "a conversão incondicional de IV" funciona verificando que os valores **já gravados no JSON da fixture** estão entre 1 e 400. A função que faz a conversão (`brapi.ts:438-447`) nunca é chamada. CT-953 afirma que a fixture tem 8 calls espaçadas em R\$ 0,25 — testa o arquivo JSON, não código.

Tautologias por asserção sobre texto de arquivo. `api-contract.test.ts` inteiro: `expect(content).not.toContain('.riskReward.')` e `expect(fs.existsSync(oldEnginePath)).toBe(false)`. São `grep` disfarçados de teste de contrato — passam para sempre e não exercitam nenhum comportamento.

Asserção que não pode falhar. `indicators.test.ts:52` verifica `riskRewardRatio >= 1.4` para `calculateRiskReward`. Provei em P6 que esse valor é **estruturalmente sempre exatamente 1,5** (o alvo é definido como `spot + risco × 1,5`), para spot 10, 35,5 ou 120. A asserção nunca reprova. Ironicamente, `trade-plan.test.ts` **tem** o teste certo (T4: R:R nunca deve ser constante) — mas só para o outro motor. A regra foi criada onde o defeito foi relatado, não onde o padrão existe.

Testes que consagram a fabricação. `fundamentals.test.ts:166-168` afirma, com comentário explícito, `expect(res.metrics.debtToEbitda.status).toBe('BOM');` // Pontuou pela dívida financeira 0.8x e `expect(res.metrics.roe.status).toBe('BOM');` // Pontuou pelo ROE normalizado de 18%. Os dois números fabricados de C2 e C5 estão gravados na suíte como comportamento esperado, sem qualquer valor de referência externo.

Cobertura ausente onde mais importa. `black-scholes.ts` é o único arquivo de domínio **sem arquivo de teste** — e é onde está o único erro de fórmula que encontrei. Nenhum teste toca `breakEven` do Iron Condor, `percentile`, ou qualquer um dos caminhos de fallback (`classifyVolatilityRegime(0,0)`, HV ausente, spot zero). Os dois casos de prejuízo em `fundamentals.test.ts` (F3, sem FCO; F9, com FCO negativo) evitam por construção exatamente a combinação que quebra o crivo.

Sobre o theta. Prova P1, `S=K=30`, `T=15/252`, `σ=30%`, `r=10,75%`, `q=3%`, contra Black-Scholes-Merton fechado calculado independentemente:

```
theta PUT  codigo = -0.018 | referencia BSM = -0.010923 | erro = -0.007077   (65% de exagero)
theta CALL codigo = -0.019 | referencia BSM = -0.017190 | erro = -0.001810
```

Duas causas em `black-scholes.ts:85-88`: o termo de juros entra com sinal negativo também para a PUT (deve ser $+ rKe^{-rT}N(-d_2)$), e o termo de dividendos $\mp qSe^{-qT}N(\pm d_1)$ está ausente nos dois. **Registro em favor do sistema:** como essa função é código morto, o erro é uma mina latente, não um número errado em tela hoje. Vira ativo no instante em que alguém religar o motor local — e por isso deve ser corrigido antes disso, não depois.

7. Fase 2 — Reconciliação com o trabalho anterior

Registro de método, primeiro. A Fase 2 pedia abrir os arquivos `AUDITORIA_*.md` da raiz. **Eles não existem** — nem na raiz, nem em subpasta, nem em qualquer commit do histórico (`git log --all --diff-filter=A --name-only | grep -i audit` devolve apenas `study-auditor.*` e `run_full_audit.js`, que são código). Não vou fingir reconciliar documentos ausentes. Reconciliei contra o laudo anterior que **de fato existe** e cumpre esse papel: `docs/ESPEC-EXECUCAO-RADAR-B3-v3.1-VERIFICACAO.md` (31/08/2026,

base 308f47a, 11 itens com placar), apoiado em docs/ESPEC-TECNICA-RADAR-B3-v2.1-CORRECOES.md, v2.3-VALIDACAO.md e WALKTHROUGH.txt.

7.1 Placar dos achados anteriores

#	Item anterior	Status anterior		Evidência atual
03	Fallback de IV anula o gate	● crítico	RESOLVIDO (com ressalva)	O bloco de reconstrução sumiu. cme-election.ts:473 é hoje <code>const atmIv = optionAnalysis?.ivAtm?.callIv;</code> — leitura direta, sem reconstrução. CT-958 e CT-958b existem e passam.
03b	Escopo: débito não deveria ser bloqueado por falta de IV (CT-960)	critério de aceite 2	INTACTO	cme-election.ts:472: <code>isCreditTarget</code> cobre ALTA \\ \\ BAIXA \\ \\ LATERAL — tudo. CT-960 não existe na suíte.
06	<code>isActionable</code> ausente nos dois componentes	● não feito	PARCIAL — e piorou de forma sutil	Ver 7.3.
07	Afordâncias de acessibilidade	⚠ parcial	PARCIAL	<code>nav-tab-*</code> : 7 ocorrências (feito) · <code>role="tabpanel"</code> : 6 (feito) · <code>aria-controls</code> : 1 de 12 abas · <code>aria-live</code> : 0. Critério de aceite 6 não cumprido.
10	<code>alternative</code> nunca preenchida	✗ meio-feito	RESOLVIDO	cme-election.ts:873 e :1234 atribuem <code>strategyResult.alternative</code> com <code>rationale</code> , após passar por <code>assertDirection</code> . CT-N06 existe e passa.
—	Decisão pendente: pesos do score	—	DECIDIDO conforme proposta	<code>rules.ts:120-125</code> = 0,50 / 0,15 / 0,20 / 0,15, exatamente a proposta da seção 6 do laudo. Consumidos em cme-election.ts:599.
—	Anexo: limpeza de .tgz	—	RESOLVIDO	.gitignore cobre *.tgz; _qa3.tgz permanece em disco, não rastreado.

Itens 01, 02, 04, 05, 08, 09 e 11, já  no laudo anterior, seguem intactos — reconfirmei 04 (diagnóstico numérico real em todos os caminhos de bloqueio, cme-election.ts:355-362, 625-632) e 05 (busca cartesiana de condor, :1324-1364).

7.2 Achados anteriores que eu não encontrei sozinho na Fase 1

Três, e por razões diferentes — vale distinguir:

- **Item 03 (fallback de IV): não encontrei porque foi corrigido.** Auditei cme-election.ts:471-516 na Fase 1 e registrei o gate como bem construído. A correção é real e eu a validei sem saber que existia — que é o melhor teste possível dela. Encontrei um **descendente** do problema, esse sim vivo: o gate chama `classifyVolatilityRegime(atmIv, hv21)` na linha 496 com um `hv21` que pode ser o 25,0 fabricado (C8). A IV foi limpa; a HV do mesmo gate não foi.
- **Item 06 (`isActionable`): passei por cima.** Li `QuoteView.tsx` e `OptionsBarriersView.tsx` procurando fabricação de número, e a condição de renderização não estava no meu foco. Só apareceu quando o laudo anterior me disse onde olhar. Registro como falha minha de varredura — e ver 7.3, porque o que estava lá era pior do que uma pendência.
- **Item 07 (acessibilidade): fora do escopo declarado.** Os quatro eixos desta auditoria são qualidade, integridade, acuracidade e segurança de dados. Não varri afordâncias de teste. Reconciliei por medição direta, não por auditoria própria.

7.3 Regressão introduzida pela correção anterior

Este é o achado mais interessante da Fase 2, e é uma regressão de tipo raro.

O laudo anterior pediu (critério de aceite 5): "`isActionableStrategy` é a única forma de decidir se uma estrutura é acionável, nos dois componentes", e propôs o teste estático CT-967c: "varredura estática — nenhuma comparação direta a 'BLOQUEADA' fora do util".

O que aconteceu:

```
isActionableStrategy criado em cme-election.ts:137
importado em QuoteView.tsx:35 e usado em QuoteView.tsx:997
importado em OptionsBarriersView.tsx:24 e usado ...
```

✓
✓
✗ NUNCA

OptionsBarriersView.tsx:531 continua com condição escrita à mão — mas **reescrita**:

```
{data.electedOptionStrategy && (data.electedOptionStrategy.status === 'AUTORIZADA'
|| data.electedOptionStrategy.status === 'EM_ANALISE')
&& data.electedOptionStrategy.legs.length > 0 && (
```

Três consequências:

1. **A varredura CT-967c passaria.** `grep "status !== 'BLOQUEADA'" src/components/` hoje devolve **zero** ocorrências. O padrão exato que o critério de aceite mandava procurar desapareceu — porque foi **reescrito em outra grafia**, não porque foi eliminado. O teste que o laudo anterior desenhava para pegar esse caso ficou cego para ele. (CT-967c, aliás, nunca chegou a ser escrito: não existe na suíte.)
2. **O defeito latente virou explícito.** Antes, `EM_ANALISE` com pernas caía no painel de autorizada por acidente de implementação. Agora a condição **inclui** `EM_ANALISE` **deliberadamente**. `EM_ANALISE` é o status de `PRECO_DEFASADO` (`cme-election.ts:299`) — defasagem de spot > 3%, o caso em que os prêmios não representam mais o mercado. A página nova renderiza essa estrutura no painel de estrutura autorizada.
3. **O import não utilizado é a impressão digital.** `isActionableStrategy` foi importado no arquivo e não chamado — o gesto de quem aplicou a correção pela metade. Um `no-unused-vars` de ESLint teria apontado em um segundo, e é exatamente a barreira automática que o projeto não tem (A9).

Isto é a **quarta ocorrência** do padrão que a "Nota de método" do laudo anterior já havia registrado três vezes: *a correção é aplicada onde o defeito foi relatado, não onde o padrão existe*. Desta vez com um agravante novo — a reescrita tornou o defeito invisível ao próprio critério de aceite.

7.4 Achados novos que nenhum laudo anterior tinha

Todos os 9 críticos e 15 altos da Seção 2 são novos. Os laudos anteriores tratam exclusivamente do motor de eleição de opções e da camada de UI/automação. Nenhum deles toca em: crivo fundamentalista, serviço de cotações globais, motor de sentimento, indicadores técnicos, proveniência de dado, segurança de rota, ou dado pessoal em repositório. É um ângulo de auditoria que ainda não tinha sido aberto — o que explica por que os problemas mais graves do sistema estavam intocados enquanto os itens de opções avançavam bem.

8. Mapa de risco: onde assumir, onde mitigar

Assumir.

- *Selic hardcoded a 10,75% (M1).* Vigente hoje é 14,50% ([Bacen](#)). Medi o impacto: em 15 DTE, 3% a 6% no preço teórico, 1-2 pontos de delta, 0,7 pp de IV. Como o motor é código morto, o risco corrente é zero. Assumir até religar o motor; corrigir junto.
- *ATR por média simples em vez de Wilder (B3).* Diferença de suavização, não de ordem de grandeza; o stop é estrutural (suporte – 0,5 ATR), não puramente ATR.
- `calculateHistoricalRSI` $O(n^2)$ (B2). 252 pontos, custo irrelevante.

- *Persistência em JSON no filesystem.* Na Vercel o disco é efêmero e os dados se perdem — mas são newsletter e contador, não decisão de capital. Assumir enquanto for hobby; mitigar ao monetizar.
- *Painel setorial hardcoded (A2).* Assumir **apenas** se ganhar tarja explícita de "exemplo ilustrativo". Sem tarja, é mitigar.

Mitigar sem negociação.

- *Todo fallback numérico silencioso (C3, C6, C7, C8, e os `|| 0.5`, `|| 100`, `|| 0.14` da UI).* Este é o risco estrutural do sistema, não uma lista de bugs. Enquanto existir, nenhum número em tela é auditável, e a nota de integridade não passa de 5 por mais que o resto melhore.
- *Contaminação do crivo fundamentalista (C1, C2, C5).* É o filtro que autoriza a operação. Comprometido, o R1/R2/R3/R4 inteiro perde valor.
- *Break-even do Iron Condor (C4).* Número errado com consequência financeira direta e correção de 20 minutos.
- *Falta de dado virando veredito (C9).* Precisa de um terceiro estado, não de um `catch`.
- *Divergência rastreador × consulta individual (A1).* O mesmo ativo, no mesmo instante, recebe estruturas diferentes nas duas telas — RSI 50 fixo e ausência do gate de volatilidade no rastreador. Isso quebra o requisito explícito de determinismo entre módulos.
- *Rotas abertas + PII versionada (A7, A8, A10).* Custo e exposição legal.
- *ESLint + CI (A9).* Barata e paga a si mesma: teria pego o import morto de 7.3.

Risco de decisão, não técnico. Enquanto `version.ts` disser "Produção / B3 Realtime" e a aba disser "Consultor IA", o sistema promete mais do que entrega. Num produto pessoal isso é ruído; num produto oferecido a terceiro sob rótulo CNPI, é exposição.

9. Plano de ação, em ordem de execução

Bloco 0 — antes de qualquer outra coisa (1 dia). Sem isto, toda correção seguinte pode ser desfeita sem ninguém notar.

1. Instalar e configurar ESLint com `no-unused-vars` e `no-restricted-syntax` proibindo literal numérico em `??/||` dentro de `src/lib/domain` e `src/lib/services`.
2. GitHub Actions com `lint` + `test` obrigatórios no push.
3. `git rm --cached data/*.json`, `/data` no `.gitignore`.

Bloco 1 — números errados em tela (2 dias). Correções pequenas, impacto direto.

4. `cme-election.ts:1476` → `breakEvenLower / breakEvenUpper`; ajustar tipo e UI. **Teste de referência:** payoff nos dois BEs ≈ 0 ; payoff no centro = crédito total.
5. `fundamentals.ts:116` → exigir `netIncomeVal > 0`. **Teste:** o caso de P4 deve voltar a REPROVADO com as duas flags.
6. `fundamentals.ts:209-211` → remover o ramo do ROE escalado por FCO.
7. `brapi.ts:326-335` → apagar o bloco `VALE3`.
8. Reescrever `fundamentals.test.ts` F8 contra números publicados no ITR da empresa, não contra a saída do código.

Bloco 2 — proveniência (1 semana). É a correção estrutural; as outras são consequência.

9. Criar `type Provenance = { source: 'MEDIDO' | 'FONTE_INDISPONIVEL'; asOf: string; provider: string }` e envelopar todo número exibido.
10. Remover **todos** os fallbacks numéricos: `options-barriers.ts:140,142,143,188,230,248`; `cme-election.ts:346`; `market-quotes.ts:63-76`; `volatility.ts:51-52`; `indicators.ts:116,139`; `trade-`

`plan.ts:86`; `study-auditor.ts:110,111,326`; `QuoteView.tsx:451,457`; `OptionsBarriersView.tsx:299`; OS ?? 25.0 das três rotas. Cada um vira `null`.

11. UI: `null` renderiza como N/D com o motivo; nunca como número.
12. `analyzeOptionPositions` deve lançar com `spot ≤ 0`; `buildOptionBarrierAlert` deve devolver `hasWalls: false` em vez de inventar muralhas.
13. Estado `INDISPONIVEL` no crivo fundamentalista, bloqueando listagem.

Bloco 3 — coerência entre módulos (3 dias).

14. Extrair a montagem do contexto de análise (`buildAnalysisContext`) para uso idêntico por `/quote`, `/options` e `/trends`. Elimina o RSI 50 fixo e restaura o gate de volatilidade no rastreador.
15. `OptionsBarriersView.tsx:531` → `isActionableStrategy(...)`, e painel próprio para `EM_ANALISE` com tarja e sem boleta. Escrever o CT-967c que faltou — mas como teste de **comportamento**, não `grep`.
16. Mover para `rules.ts` os thresholds duplicados (M3) e implementar ou remover as regras `NORMALIZATION` mortas (M6).

Bloco 4 — rótulos e segurança (3 dias).

17. Renomear "Consultor IA" para "Manual Interativo"; ou plugar um LLM de verdade.
18. Corrigir `environment` em `version.ts`; derivar `build` do commit.
19. Tarja de "exemplo ilustrativo" no painel setorial, ou removê-lo.
20. Pilares 4 e 5 do Termômetro: plugar DI (B3/Bacen) e fluxo estrangeiro (boletim B3), ou renomear e reduzir o peso — 45% do índice não pode repousar sobre o sinal do EWZ.
21. Rate limit por IP real (`@vercel/kv` ou similar) nas rotas; teto rígido em `/trends?limit=`; converter `/analytics/track` em POST.

Bloco 5 — cobertura (contínuo).

22. `black-scholes.test.ts` com valores de referência externos; corrigir o theta da PUT e o termo de dividendos.
23. Substituir `api-contract.test.ts` e CT-952/CT-953 por testes de comportamento.
24. Um teste por caminho de fallback removido, afirmando `null`.

Esforço estimado: 3 a 4 semanas de trabalho focado (≈ 100-130 h), sendo o Bloco 2 metade disso. Os Blocos 0 e 1 somam 3 dias e já retiram os erros de maior consequência financeira.

10. Recomendação final

Remediar. Não reescrever.

A justificativa é específica, não diplomática: o que está errado neste sistema é **periférico e removível**, e o que está certo é **o que seria caro refazer**. O motor de eleição de estruturas, a matriz de decisão, o cálculo de payoff, a busca cartesiana de Iron Condor, a arquitetura de bloqueio com diagnóstico numérico — tudo isso está correto, foi verificado por execução, e representa a maior parte do valor intelectual do projeto. Uma reescrita jogaria fora exatamente essa parte e reintroduziria erros que já foram encontrados e corrigidos ao longo de v2.1, v2.2, v2.3, v3.0 e v3.1.

Os defeitos, ao contrário, são de uma classe só, repetida: **ausência de tipo de proveniência**. Catorze constantes de fallback existem porque a assinatura das funções não tem como dizer "não sei". Introduza esse tipo e a maior parte dos críticos deixa de ser corrigível-um-a-um para virar corrigível-de-uma-vez, com o compilador apontando cada lugar que falta. É por isso que o Bloco 2 é o item caro e o item certo.

Duas condições, porém, sem as quais a remediação repete o ciclo:

Primeira: as barreiras automáticas antes das correções, não depois. A Seção 7.3 mostra o custo de não tê-las — uma correção parcial que se tornou invisível ao próprio critério de aceite que a auditava, e um import morto que qualquer linter apontaria. Quatro rodadas de auditoria encontraram o mesmo padrão de "corrigido onde foi relatado, não onde existe". Isso não se resolve com mais uma rodada de auditoria; resolve-se com `grep` obrigatório no CI.

Segunda: nenhum número em tela sem origem declarada. Este é o critério de aceite único que governa todos os outros. Enquanto um usuário não puder olhar para "HV21: 24,5%" e saber, pela tela, se aquilo foi medido ou preenchido, o sistema não é auditável — e um terminal quantitativo que não é auditável não é um terminal quantitativo, é uma opinião com formatação boa.

Até que os Blocos 0, 1 e 2 estejam concluídos e verificados por teste adversarial de fora, **a recomendação é não usar os números deste sistema para dimensionar posição real**, com a exceção parcial da estrutura de opções eleita — cujos strikes, prêmios e crédito vêm de dado real da BRAPI e cuja matemática validei —, e mesmo essa condicionada a conferir os prêmios das duas pontas no book antes de enviar a ordem, como o próprio sistema já adverte quando detecta defasagem.

Anexo — como reproduzir as provas

Os três scripts de prova (`probe1.ts`, `probe2.ts`, `probe3.ts`) importam os motores de produção sem modificação e rodam com `npx tsx`. Cobrem, nesta ordem: theta contra BSM fechado (P1), precisão do solver de IV (P2), fallback de volatilidade (P3), crivo fundamentalista com prejuízo (P4/P4b/P4c), barreiras sem fonte (P5), percentil constante (P5b), R:R tautológico (P6), ATR e volume com histórico curto (P7), break-even do Iron Condor (P8), feed externo indisponível (P9), termômetro sobre dado fabricado (P10) e impacto da Selic hardcoded (P11/P12).

Fontes externas consultadas: taxa Selic meta vigente — [Banco Central do Brasil](#).

Documentos internos lidos apenas na Fase 2: `docs/ESPEC-EXECUCAO-RADAR-B3-v3.1-VERIFICACAO.md`, `docs/ESPEC-TECNICA-RADAR-B3-v2.1-CORRECOES.md`, `docs/ESPEC-TECNICA-RADAR-B3-v2.3-VALIDACAO.md`, `WALKTHROUGH.txt`, `DOCUMENTO_MUDANCAS_HOJE.md`.